

良いコード・悪いコードから学ぶ Swift デザインパターン

佐藤 晶彦 / @akkiee76

はじめに

「良いコードを書きたい！」これは、エンジニアなら誰でも思うことではないでしょうか。私も長年エンジニアをしています、言語やフレームワークの進化に伴い「良いコード」に対する探究は今も続いています。

では、「良いコード」とはどのようなコードでしょうか。保守性の高いコード？それとも可読性の高いコードでしょうか。実際、良いコードはエンジニアの価値観にも依存するため、バックグラウンドやキャリアによって捉え方が変わって来るのではないのでしょうか。しかし、「一定の品質を保つことができる堅牢なコード」は Swift に限らず様々な言語において良いコードの要素だと私は考えております。この記事では、オブジェクト指向の観点から実装例を踏まえて、基本的な Swift のデザインパターンを紹介します。

1. 完全コンストラクタ (Complete Constructor)

完全コンストラクタとは、オブジェクト指向プログラミングにおいて、インスタンスを生成する際に、そのオブジェクトのすべての property を初期化するためのコンストラクタのことです。これによりオブジェクトが不完全な状態で存在することを防ぎ、初期化後の値を保証することができます。また、完全コンストラクタには以下の特徴があります。

- オブジェクトの中で各 property の値が保証される
- オブジェクトの各 property を参照時に予期しない nil 参照を回避できる
- nil を考慮した例外処理の設計が不要になる

Swift の class や struct では `init()` の自動生成により各 property の値が初期化されるため、完全コンストラクタを容易に実現できるという特徴があります。

```
struct User {  
    let name: String  
    let createdAt: Date  
    let updatedAt: Date  
  
    init(name: String, createdAt: Date, updatedAt: Date) {  
        self.name = name  
    }  
}
```

```

        self.createdAt = createdAt
        self.updatedAt = updatedAt
    }
}

```

また、外部から変更が想定される property を含む場合は、actor を使用して一つの property にアクセスが同時に行われないようにしましょう。actor を指定していれば、データ競合が生じる場合にコンパイラ側で検知することが可能です。

```

actor User {
    var name: String // User外から変更される
    let createdAt: Date

    init(name: String, createdAt: Date) {
        self.name = name
        self.createdAt = createdAt
    }
}

```

次に、以下のコードは prepare() による画面遷移時の実装例ですが、遷移時に userDetails を代入しています。このケースでは ProfileViewController の userDetails は var を指定する必要があるため、意図しない再代入のリスクが生じます。

```

final class UserViewController: UIViewController {
    // 省略
    override func prepare(for segue: UIStoryboardSegue, sender: Any?) {
        if segue.identifier == "identifier" {
            let vc = segue.destination as! ProfileViewController
            vc.userDetail = response.userDetail
        }
    }
}

```

このようなケースでは、UIStoryboardSegue > instantiateInitialViewController を使用することで、ProfileViewController の初期化時に userDetails を代入することができるため let 指定にすることができます。これによって意図せず値が再代入されるリスクがなくなり、コードの堅牢性を向上させることが可能です。

```

final class ProfileViewController: UIViewController {
    private let userDetail: UserDetails
    // 省略
    init?(coder: NSCoder, userDetail: UserDetails) {
        self.userDetail = userDetail
        super.init(coder: coder)
    }
}

```

```

}

final class UserViewController: UIViewController {
    // 省略
    func showProfileViewController() {
        let storyboard = UIStoryboard(name: "Profile", bundle: nil)
        let vc = storyboard.instantiateInitialViewController { coder in
            ProfileViewController(coder: coder, userDetails:
self.response.userDetail)
        }
        navigationController?.pushViewController(vc!, animated: true)
    }
}

```

このように、完全コンストラクタで初期化時に property の値を設定することで意図しない値の再代入を防ぐことができ、コードの堅牢性を向上させることができます。

2. 値オブジェクト (Value Object)

値オブジェクトとは値を型とする設計パターンで、以下の特徴があります。

- 計測、定量化、説明を責務とする
- 状態の不変性を維持する
- 値の等価性を判定できる
- 副作用のない振る舞いを提供する

アプリ開発では、氏名、郵便番号、金額、税率など様々な値を扱う場面において、これらの操作をオブジェクトに集約することでより堅牢な設計にすることができます。

では、以下のコードを見てみましょう。

```

struct User {
    let firstName: String
    let lastName: String

    var fullName: String {
        return "\(firstName) \(lastName)"
    }

    func compareName(user: User) -> Bool {
        return firstName == user.firstName && lastName == user.lastName
    }
}

```

User はユーザに関する情報を持つ struct ですが、fullName や compare() の名前に関するロジックが実装されており、User の関心ごとがやや多くなっている印象です。そこで、この名前を値

オブジェクトに変更してみましょう。名前固有の `fullName` や `Equatable` で等価性が判定できるようになり、`User` から `Name` に関するロジックを移行することで凝集性を高めることができます。

```
struct Name: Equatable {
    let firstName: String
    let lastName: String

    var fullName: String {
        return "\(firstName) \(lastName)"
    }

    init(firstName: String, lastName: String) {
        self.firstName = firstName
        self.lastName = lastName
    }
}

struct User {
    let name: Name
    // 省略
}

extension User {
    func compareName(user: User) -> Bool {
        return self.name == user.name
    }
}
```

このように、値オブジェクトに変更することで高凝集な設計にすることができます。また、前述の完全コンストラクタと併用することでさらに堅牢な設計を目指すことも可能です。

3. ストラテジーパターン (Strategy Pattern)

ストラテジーパターンとは、アルゴリズムを動的に切り替える必要がある場合、それらを外部からカプセル化することで、必要なケースに応じてアルゴリズムを使い分けられるようにするデザインパターンです。アルゴリズムを実行する部分とそれを利用する部分が分離されるため、変更容易性の高い設計を実現することができます。これにより、ロジックの独立性とテスト容易性を向上させることができます。

一方、ポリシーパターンは、ストラテジーパターンと同様にアルゴリズムや振る舞いをカプセル化するためのデザインパターンですが、ビジネスルールや方針 (ポリシー) を定義し、これらのポリシーを適用するためのインタフェースを提供するデザインパターンです。これにより任意のモジュールに複数のポリシーを適用することができるデザインパターンです。

では、以下のコードを見てみましょう。

```

struct UserRepository {
    let requestType: RequestType

    init(requestType: RequestType) {
        self.requestType = requestType
    }

    var path: String {
        switch requestType {
        case .profile:
            return "/user/v1/profile"
        case .activity:
            return "/user/v2/activity"
            // 以降、RequestTypeのcaseが続く...
        }
    }

    func getParameter(value1: String, value2: String) -> [[String: String]] {
        switch requestType {
        case .profile:
            return [:"key1": value1], [:"key2": value2]]
        case .activity:
            return [:"key3": value1], [:"key4": value2]]
            // 以降、RequestTypeのcaseが続く...
        }
    }

    func fetchData(value1: String, value2: String) async throws -> Data {
        let parameter = getParameter(value1: value1, value2: value2)
        // リクエスト処理
    }
}

```

UserRepository は requestType に応じてユーザに関連する情報をAPIで取得するための機能です。getPath() と getParameter() の中ではそれぞれ switch 文で条件分岐が実装されて冗長になっています。今後さらに case が増えると UserRepository のコードが肥大化していきます。そこで、ストラテジーパターンで再設計してみましょう。

```

protocol UserRepositoryProtocol {
    var path: String { get }
    func getParameter(value1: Any, value2: Any) -> [[String: Any]]
    func fetchData(value1: String, value2: String) async throws -> Data
}

```

まず Protocol を定義し、UserRepository から各 RequestType 別に責務分割をします。

```

struct UserProfileRepository: UserRepositoryProtocol {
    var path: String {
        return "/user/v1/profile"
    }

    func getParameter(value1: Any, value2: Any) -> [[String : Any]] {
        return [{"key1": value1}, {"key2": value2}]
    }

    func fetchData(value1: String, value2: String) async throws -> Data {
        let parameter = getParameter(value1: value1, value2: value2)
        // リクエスト処理
    }
}

```

```

struct UserActivityRepository: UserRepositoryProtocol {
    var path: String {
        return "/user/v2/activity"
    }

    func getParameter(value1: Any, value2: Any) -> [[String : Any]] {
        return [{"key3": value1}, {"key4": value2}]
    }

    func fetchData(value1: String, value2: String) async throws -> Data {
        let parameter = getParameter(value1: value1, value2: value2)
        // リクエスト処理
    }
}

```

次に protocol に準拠し、Repository を各責務ごとに分離・再設計することで、ストラテジーパターンに設計変更することができました。Repository のような汎用クラスで冗長になることが想定される場合、ストラテジーパターンによる設計が有効です。また、protocol に準拠しない場合はコンパイルエラーになるので、必須関数の定義漏れ防止の観点からも有効な設計パターンといえます。このように、ストラテジーパターンを利用することで、絡み合う複雑な実装をシンプルに再設計することができます。

4. ファーストクラスコレクション (First Class Collection)

ファーストクラスコレクションとは、Collection (List、Set、Map) 自体をオブジェクトとして扱う設計方法で、Collection に関連する固有のロジックを隔離することができます。それによって、責務の分離、再利用性の向上、テスト容易性の向上が見込めます。

では、以下のコードを見てみましょう。

```
func addItem(_ item: Item) {
    if items.count == 10 {
        fatalError("already max count.")
    }
    if items.firstIndex(where: { $0.id == item.id }) == nil {
        return
    }
    items.append(item)
}
```

この addItem 関数は自身の items に要素を追加する関数ですが、要素数バリデーション、重複チェック、要素の追加の責務が同じ関数に含まれています。また、直接 items に要素が追加されているため、副作用が生じる可能性があります。そこで、ファーストクラスコレクションでそれぞれの責務を分離し、副作用が生じないオブジェクトに変更してみましょう。

```
struct ItemCollection {
    private var items: [Item] = []
    private let maxCount = 10

    init(items: [Item]) {
        self.items = items
    }

    func add(item: Item) -> ItemCollection {
        if isMaxCount() {
            fatalError("already max count.")
        }
        if contains(item: item) {
            return self
        }
        var newItems = items
        newItems.append(item)
        return ItemCollection(items: newItems)
    }

    func contains(item: Item) -> Bool {
        return items.firstIndex(where: { $0.id == item.id }) != nil
    }

    func isMaxCount() -> Bool {
        return items.count == maxCount
    }
}
```

ItemCollection を作成し、要素数バリデーション、重複チェック、要素の追加を関数化しそれぞれの責務を分離しました。要素を追加する際は直接 items が変更されないため、副作用を避けることができます。

このように、ファーストクラスコレクションで設計することによって、コレクションをより安全に扱うことができます。

5. スプラウトクラス (Sprout Class)

スプラウトクラスとは、既存のクラスの機能を拡張するための手法の一つで、変更に必要な機能を別のクラスとして切り出し、既存のクラスから利用する手法です。それにより既存クラスに直接の変更を加えることなく新しい機能を追加することができるため、既存クラスへの影響を最小にすることができます。

では、以下のコードを見てみましょう。既存の通常課金を責務とする Payment にサブスクリプション機能が追加されたコードです。

```
struct Payment {
    private let identifier: String

    init(identifier: String) {
        self.identifier = identifier
    }

    func processPayment() {
        // 購入処理
    }

    // サブスクリプション関連のプロパティを追加
    private var subscriptionType: SubscriptionType?
    private var startedAt: Date?

    // サブスクリプション関連のメソッドを追加
    func processSubscription() {
        if let subscriptionType = subscriptionType, let startedAt = startedAt {
            if !enableSubscription(subscriptionType: subscriptionType, startedAt:
startedAt) {
                fatalError("not available subscription.")
            }
            // その他の処理
        }
        processPayment()
    }

    func enableSubscription(subscriptionType: SubscriptionType, startedAt:
Date) -> Bool {
        // サブスクリプションの有効性をチェックする処理
        return enable
    }
}
```

```
}
```

既存の Payment にサブスクリプションの処理が追加されて、自身の責務が増えています。Payment に実装されている通常購入処理 processPayment() とサブスクリプション購入処理 processSubscription() を定義することで関連ロジックも増え、コードの複雑性も上がります。そこで、このサブスクリプションの機能をスプラウトクラスを利用して Payment からその責務を分離してみましょう。

```
protocol PaymentProtocol {
    var identifier: String { get }
    func processPayment()
}

extension PaymentProtocol {
    func processPayment() {
        // 購入処理
    }
}

struct Payment: PaymentProtocol {
    var identifier: String

    init(identifier: String) {
        self.identifier = identifier
    }
}
```

```
struct SubscriptionPayment: PaymentProtocol {
    var identifier: String
    let subscriptionType: SubscriptionType
    let startedAt: Date

    init(identifier: String, subscriptionType: SubscriptionType, startedAt:
Date) {
        self.identifier = identifier
        self.subscriptionType = subscriptionType
        self.startedAt = startedAt
    }

    func processSubscription() {
        if !enable() {
            fatalError("not available subscription.")
        }
        // その他の処理
        processPayment()
    }
}
```

```
func enable() -> Bool {  
    // サブスクリプションが有効化チェックする処理  
    return enable  
}  
}
```

Payment と SubscriptionPayment の共通の処理である processPayment() をそれぞれ共通で利用できるように protocol を定義し extension 側にロジックを移行しました。そして、SubscriptionPayment に Subscription に関連するロジックを追加して責務を分離することで、Payment への影響と肥大化を回避し SubscriptionPayment のロジックの独立性とテスト用意性を向上させることができました。

このように、スプラウトクラスを利用することで、既存実装に直接の変更を加えることなく安全に新しい機能を追加でき、同時にロジックの独立性とテスト容易性を向上させることができます。

さいごに

オブジェクト指向をベースとして基本的な Swift デザインパターンを紹介させて頂きました。この記事が開発者のスキル向上とプロジェクトの成功に貢献する貴重な情報源となることを願っています。なお、本記事は情報の提供を目的としておりますので、本記事を利用した運用は必ずご自身の自己責任と判断によって行なってください。

感想・フィードバック

本記事の感想やフィードバックを X アカウント: @akkiee76 にてお待ちしております。お気軽にどんどん送ってください。ここが良かった、ここが分かりにくかったなど今後の執筆の参考にできればと思います。また、本記事で扱ったサンプルコードは以下の GitHub リポジトリにて公開しております。改善点や疑問などありましたら、Pull Request や Issue の作成もお待ちしております。

- <https://github.com/akkie76/swift-design-patterns>